COL380 A3

Abhishek Amrendra Kumar

April 2026

1 Results

Processes	Time (sec)	Speedup
1	49.61	1.00
2	34.27	1.45
4	22.76	2.18
8	13.76	3.61
16	8.99	5.51

2 Speedup Graph

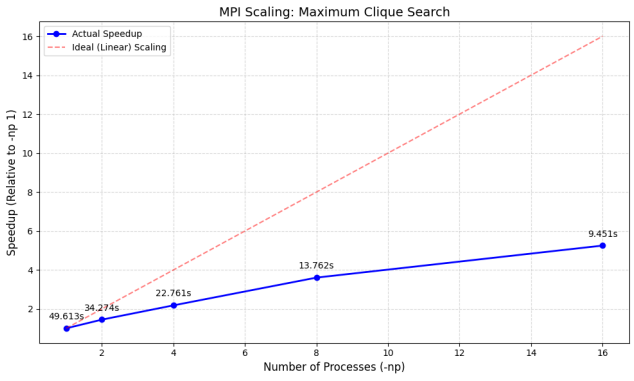

Figure 1: Speedup vs Number of Processes on the test graph

Observation:

- Speedup increases with number of processes
- Sublinear scaling due to load imbalance and limited parallelism

3 Key Data Structures

Bitset Adjacency

```
vector<bitset<MAXN>> adj;
```

Each vertex stores neighbors as a bitset, enabling fast clique updates via: $C_{\text{next}} = C_{\text{cand}} \& adj[v]$. This avoids explicit iteration and makes intersection operations very efficient.

Bitset Candidate Set

```
bitset<MAXN> cand;
```

Used to maintain valid extensions of the current clique. Supports fast masking and pruning.

Global Order (Structural Bound) Vertices sorted by decreasing profit:

```
vector<int> global_order;
```

Used in greedy coloring to compute a tight structural upper bound.

Ratio Order (Knapsack Bound) Vertices sorted by $\frac{p(v)}{c(v)}$:

```
vector<int> ratio_order;
```

Used for fractional knapsack bound to estimate maximum achievable profit.

Coloring Structures

```
bitset<MAXN> colorSets[MAXN];
int colorMax[MAXN];
```

Implements greedy coloring; bound is sum of maximum profit per color class.

4 MPI Parallelization

Work Distribution Search space split at top level:

```
for (int i = rank; i < N; i += size)
```

Each process explores independent subtrees.

Data Broadcast Process 0 reads input and shares using `MPI_Bcast` (graph + vertices).

Local Computation Each process runs Branch Bound independently with its own best solution. No communication during search.

Reduction

```
MPI_Allreduce(..., MPI_MAXLOC)
```

Finds global best profit and the process that achieved it.

Collection Winning process sends clique to rank 0 using `MPI_Send/Recv`.

Speedup Achieved via parallel exploration of disjoint branches with minimal communication overhead.